

vLLM专题（七）-支持的模型列表

原创

AI专题精讲

已于 2025-03-19 11:24:21 修改

阅读量564

收藏

点赞数

CC 4.0 BY-SA版权

分类专栏：

大模型专题系列

文章标签：

人工智能

大模型专题系列 专栏收录该内容

您已是超级会员，正在免费阅读会员专享内容

查看更多超级会员权益

vLLM支持跨多种任务的生成式和池化模型。如果一个模型支持多个任务，您可以通过`-task`参数来设置任务。

对于每个任务，我们列出了已在vLLM中实现的模型架构。每个架构旁边，我们列出了一些使用该架构的流行模型。

1. 加载模型

1.1 HuggingFace 模型库

默认情况下，vLLM 会从 HuggingFace (HF) Hub 加载模型。

要确定某个模型是否受支持，您可以检查 HF 仓库中的 `config.json` 文件。如果“architectures”字段包含以下列出的模型架构，那么理论上该模型应该是受支持的。

提示

检查您的模型在运行时是否真正受支持的最简单方法是运行以下程序：

```
from vllm import LLM

# For generative models (task=generate) only
llm = LLM(model=..., task="generate") # Name or path of your model
output = llm.generate("Hello, my name is")
print(output)

# For pooling models (task={embed,classify,reward,score}) only
llm = LLM(model=..., task="embed") # Name or path of your model
output = llm.encode("Hello, my name is")
print(output)
```

¥49.90
¥99.00

111 篇文章

已订阅

8折续

如果 vLLM 成功返回文本（对于生成式模型）或隐藏状态（对于池化模型），则表明您的模型是受支持的。

否则，请参考 添加新模型部分，了解如何在 vLLM 中实现您的模型。或者，您可以在 GitHub 上提交问题，请求 vLLM 支持。

1.2 Transformers 后备方案

在合并 Pull Request #11330 之后，vLLM 可以回退到 transformers 中可用的模型。目前这并非对所有模型都有效，但大多数解码器语言模型已支持，视觉语言模型的支持也在计划中！

要检查后端是否为 transformers，您可以简单地执行以下操作：

```
from vllm import LLM
llm = LLM(model=..., task="generate") # Name or path of your model
llm.apply_model(lambda model: print(model.__class__))
```

如果显示为 **TransformersModel**，则意味着它是基于 transformers 的！

1.3 支持的功能

1.3.1 LORA 和量化

这两者目前尚未支持！请务必提交一个问题，我们将与 transformers 团队一起努力解决！

通常，transformers 模型通过 `load_adapters` API 加载权重，这依赖于 PEFT。我们需要做一些工作来使用此 API（目前这会导致某些权重未被标记为已加载）或相应地替换模块。

以下是实现方式的一些提示：

```
class TransformersModel(nn.Module, SupportsLoRA):
    def __init__(*):
        ...
        self.model.load_adapter(vllm_config.load_config.model_loader_extra_config["qlora_adapter_name_or_path"])
```

问题在于你需要指定支持的 Lora 层，而我们理想情况下希望能够加载检查点中的所有内容！

1.3.2 远程代码

这意味着，任何在模型库中能够在 transformers 中使用 `trust_remote_code=True` 并正确实现注意力机制的模型都可以在生产环境中使用！

```
from vllm import LLM
llm = LLM(model=..., task="generate", trust_remote_code=True) # Name or path of your model
llm.apply_model(lambda model: print(model.__class__))
```

一个模型只需要以下两点：

```
from transformers import PreTrainedModel
from torch import nn

class MyAttention(nn.Module):

    def forward(self, hidden_states, **kwargs): # <- kwargs are required

        ...
        attention_interface = attention_interface = ALL_ATTENTION_FUNCTIONS[self.config._attn_implementation]
        attn_output, attn_weights = attention_interface(
            self,
            query_states,
            key_states,
            value_states,
            **kwargs,
        )
        ...

class MyModel(PreTrainedModel):
    _supports_attention_backend = True
```

后台发生的事情如下：

1. 配置被加载。
2. 从 auto_map 加载 MyModel 的 Python 类，并检查模型是否支持 _supports_attention_backend。
3. 使用 TransformersModel 后端。参见 /model_executors/models/transformers，它利用了 self.config._attn_implementation = “vllm”，因此需要使用 ALL_ATTENTION_FUNCTION。

就这样！

1.4 ModelScope

要使用来自 ModelScope 的模型而不是 HuggingFace Hub，请设置一个环境变量：

```
export VLLM_USE_MODELSCOPE=True
```

并且使用 `trust_remote_code=True`。

```
from vllm import LLM

llm = LLM(model=..., revision=..., task=..., trust_remote_code=True)

# For generative models (task=generate) only
output = llm.generate("Hello, my name is")
print(output)

# For pooling models (task={embed, classify, reward, score}) only
output = llm.encode("Hello, my name is")
print(output)
```

2. 仅文本语言模型的列表

2.1 生成模型

请查看此页面以获取有关如何使用生成模型的更多信息。

2.1.1 文本生成(–task generate)

Architecture	Models	Example HF Models	LoRA
<code>AquilaForCausalLM</code>	Aquila, Aquila2	<code>BAAI/Aquila-7B</code> , <code>BAAI/AquilaChat-7B</code> , etc.	✓
<code>ArcticForCausalLM</code>	Arctic	<code>Snowflake/snowflake-arctic-base</code> , <code>Snowflake/snowflake-arctic-instruct</code> , etc.	
<code>BaiChuanForCausalLM</code>	Baichuan2, Baichuan	<code>baichuan-inc/Baichuan2-13B-Chat</code> , <code>baichuan-inc/Baichuan-7B</code> , etc.	✓
<code>BloomForCausalLM</code>	BLOOM, BLOOMZ, BLOOMChat	<code>bigscience/bloom</code> , <code>bigscience/bloomz</code> , etc.	
<code>BartForConditionalGeneration</code>	BART	<code>facebook/bart-base</code> , <code>facebook/bart-large-cnn</code> , etc.	
<code>ChatGLMModel</code>	ChatGLM	<code>THUDM/chatglm2-6b</code> , <code>THUDM/chatglm3-6b</code> , etc.	✓
<code>CohereForCausalLM</code> , <code>Cohere2ForCausalLM</code>	Command-R	<code>CohereForAI/c4ai-command-r-v01</code> , <code>CohereForAI/c4ai-command-r7b-12-2024</code> , etc.	✓
<code>DbrxForCausalLM</code>	DBRX	<code>databricks/dbrx-base</code> , <code>databricks/dbrx-instruct</code> , etc.	
<code>DeciLMForCausalLM</code>	DeciLM	<code>Deci/DeciLM-7B</code> , <code>Deci/DeciLM-7B-instruct</code> , etc.	
<code>FalconForCausalLM</code>	DeepSeek	<code>deepseek-ai/deepseek-v2.1-72b-chat</code> , etc.	

DeepseekForCausalLM	DeepSeek	deepseek-ai/deepseek-11m-67b-base , deepseek-ai/deepseek-11m-7b-chat etc.	
DeepseekV2ForCausalLM	DeepSeek-V2	deepseek-ai/DeepSeek-V2 , deepseek-ai/DeepSeek-V2-Chat etc.	
DeepseekV3ForCausalLM	DeepSeek-V3	deepseek-ai/DeepSeek-V3-Base , deepseek-ai/DeepSeek-V3 etc.	
ExaoneForCausalLM	EXAONE-3	LGAI-EXAONE/EXAONE-3.0-7.8B-Instruct , etc.	✓
FalconForCausalLM	Falcon	tiiuae/falcon-7b , tiiuae/falcon-40b , tiiuae/falcon-rw-7b , etc.	
FalconMambaForCausalLM	FalconMamba	tiiuae/falcon-mamba-7b , tiiuae/falcon-mamba-7b-instruct , etc.	✓
GemmaForCausalLM	Gemma	google/gemma-2b , google/gemma-7b , etc.	✓
Gemma2ForCausalLM	Gemma2	google/gemma-2-9b , google/gemma-2-27b , etc.	✓
GLMForCausalLM	GLM-4	THUDM/glm-4-9b-chat-hf , etc.	✓
GPT2LMHeadModel	GPT-2	gpt2 , gpt2-xl , etc.	
GPTBigCodeForCausalLM	StarCoder, SantaCoder, WizardCoder	bigcode/starcoder , bigcode/gpt_bigcode-santacoder , WizardLM/WizardCoder-15B-V1.0 , etc.	✓
GPTJForCausalLM	GPT-J	EleutherAI/gpt-j-6b , nomic-ai/gpt4all-j , etc.	
GPTNeoXForCausalLM	GPT-NeoX, Pythia, OpenAssistant, Dolly V2, StableLM	EleutherAI/gpt-neox-20b , EleutherAI/pythia-12b , OpenAssistant/oasst-sft-4-pythia-12b-epoch-3.5 , databricks/dolly-v2-12b , stabilityai/stablelm-tuned-alpha-7b , etc.	
GraniteForCausalLM	Granite 3.0, Granite 3.1, PowerLM	ibm-granite/granite-3.0-2b-base , ibm-granite/granite-3.1-8b-instruct , ibm/PowerLM-3b , etc.	✓
GraniteMoeForCausalLM	Granite 3.0 MoE, PowerMoE	ibm-granite/granite-3.0-1b-a400m-base , ibm-granite/granite-3.0-3b-a800m-instruct , ibm/PowerMoE-3b , etc.	✓
GritLM	GritLM	parasail-ai/GritLM-7B-v1lm .	✓
InternLMForCausalLM	InternLM	internlm/internlm-7b , internlm/internlm-chat-7b , etc.	✓

InternLM2ForCausalLM	InternLM2	internlm/internlm2-7b, internlm/internlm2-chat-7b, etc.	✓
InternLM3ForCausalLM	InternLM3	internlm/internlm3-8b-instruct, etc.	✓
JAISLMHeadModel	Jais	inceptionai/jais-13b, inceptionai/jais-13b-chat, inceptionai/jais-30b-v3, inceptionai/jais-30b-chat-v3, etc.	
JambaForCausalLM	Jamba	ai21labs/AI21-Jamba-1.5-Large, ai21labs/AI21-Jamba-1.5-Mini, ai21labs/Jamba-v0.1, etc.	✓
LlamaForCausalLM	Llama 3.1, Llama 3, Llama 2, LLaMA, Yi	meta-llama/Meta-Llama-3.1-405B-Instruct, meta-llama/Meta-Llama-3.1-70B, meta-llama/Meta-Llama-3-70B-Instruct, meta-llama/Llama-2-70b-hf, 01-ai/Yi-34B, etc.	✓
MambaForCausalLM	Mamba	state-spaces/mamba-130m-hf, state-spaces/mamba-790m-hf, state-spaces/mamba-2.8b-hf, etc.	
MiniCPMForCausalLM	MiniCPM	openbmb/MiniCPM-2B-sft-bf16, openbmb/MiniCPM-2B-dpo-bf16, openbmb/MiniCPM-S-1B-sft, etc.	✓
MiniCPM3ForCausalLM	MiniCPM3	openbmb/MiniCPM3-4B, etc.	✓
MistralForCausalLM	Mistral, Mistral- Instruct	mistralai/Mistral-7B-v0.1, mistralai/Mistral-7B-Instruct-v0.1, etc.	✓
MixtralForCausalLM	Mixtral-8x7B, Mixtral-8x7B-	mistralai/Mixtral-8x7B-v0.1, mistralai/Mixtral-8x7B-Instruct-v0.1,	✓
	Instruct	mistral-community/Mixtral-8x22B-v0.1, etc.	
MPTForCausalLM	MPT, MPT- Instruct, MPT- Chat, MPT- StoryWriter	mosaicml/mpt-7b, mosaicml/mpt-7b-storywriter, mosaicml/mpt-30b, etc.	
NemotronForCausalLM	Nemotron-3, Nemotron-4, Minitron	nvidia/Minitron-8B-Base, mgoin/Nemotron-4-340B-Base-hf-FP8, etc.	✓
OLMoForCausalLM	OLMo	allenai/OLMo-1B-hf, allenai/OLMo-7B-hf, etc.	

OLMo2ForCausalLM	OLMo2	allenai/OLMo2-7B-1124 , etc.	
OLMoEForCausalLM	OLMoE	allenai/OLMoE-1B-7B-0924 , allenai/OLMoE-1B-7B-0924-Instruct , etc.	✓
OPTForCausalLM	OPT, OPT-IML	facebook/opt-66b , facebook/opt-impl-max-30b , etc.	
OrionForCausalLM	Orion	OrionStarAI/Orion-14B-Base , OrionStarAI/Orion-14B-Chat , etc.	
PhiForCausalLM	Phi	microsoft/phi-1_5 , microsoft/phi-2 , etc.	✓
Phi3ForCausalLM	Phi-4, Phi-3	microsoft/Phi-4 , microsoft/Phi-3-mini-4k-instruct , microsoft/Phi-3-mini-128k-instruct , microsoft/Phi-3-medium-128k-instruct , etc.	✓
Phi3SmallForCausalLM	Phi-3-Small	microsoft/Phi-3-small-8k-instruct , microsoft/Phi-3-small-128k-instruct , etc.	
PhiMoEForCausalLM	Phi-3.5-MoE	microsoft/Phi-3.5-MoE-instruct , etc.	✓
PersimmonForCausalLM	Persimmon	adept/persimmon-8b-base , adept/persimmon-8b-chat , etc.	
QwenLMHeadModel	Qwen	Qwen/Qwen-7B , Qwen/Qwen-7B-Chat , etc.	✓
Qwen2ForCausalLM	QwQ, Qwen2	Qwen/QwQ-32B-Preview , Qwen/Qwen2-7B-Instruct , Qwen/Qwen2-7B , etc.	✓
Qwen2MoeForCausalLM	Qwen2MoE	Qwen/Qwen1.5-MoE-A2.7B , Qwen/Qwen1.5-MoE-A2.7B-Chat , etc.	
StableLmForCausalLM	StableLM	stabilityai/stablelm-3b-4e1t , stabilityai/stablelm-base-alpha-7b-v2 , etc.	
StarCoder2ForCausalLM	StarCoder2	bigcode/starcoder2-3b , bigcode/starcoder2-7b , bigcode/starcoder2-15b , etc.	
SolarForCausalLM	Solar Pro	upstage/solar-pro-preview-instruct , etc.	✓
TeleChat2ForCausalLM	TeleChat2	Tele-AI/TeleChat2-3B , Tele-AI/TeleChat2-7B , Tele-AI/TeleChat2-35B , etc.	✓
XverseForCausalLM	XVERSE	xverse/XVERSE-7B-Chat , xverse/XVERSE-13B-Chat , xverse/XVERSE-65B-Chat , etc.	✓

注意
目前，vLLM 的 ROCm 版本仅支持 Mistral 和 Mixtral，且上下文长度限制为 4096。

2.2 池化模型

请查看此页面以获取有关如何使用池化模型的更多信息。

重要提示
由于某些模型架构同时支持生成任务和池化任务，您应该明确指定任务类型，以确保模型以池化模式而不是生成模式使用。

2.2.1 文本嵌入（--task embed）

Architecture	Models	Example HF Models	LoRA	PP
<code>BertModel</code>	BERT-based	<code>BAAI/bge-base-en-v1.5</code> , etc.		
<code>Gemma2Model</code>	Gemma2-based	<code>BAAI/bge-multilingual-gemma2</code> , etc.		✓
<code>GritLM</code>	GritLM	<code>parasail-ai/GritLM-7B-v1lm</code>	✓	✓
<code>LlamaModel</code> , <code>LlamaForCausalLM</code> , <code>MistralModel</code> , etc.	Llama-based	<code>intfloat/e5-mistral-7b-instruct</code> , etc.	✓	✓
<code>Qwen2Model</code> , <code>Qwen2ForCausalLM</code>	Qwen2-based	<code>ssmits/Qwen2-7B-Instruct-embed-base</code> (see note), <code>Alibaba-NLP/gte-Qwen2-7B-instruct</code> (see note), etc.	✓	✓
<code>RobertaModel</code> , <code>RobertaForMaskedLM</code>	RoBERTa-based	<code>sentence-transformers/all-roberta-large-v1</code> , <code>sentence-transformers/all-roberta-large-v1</code> , etc.		
<code>XLRobertaModel</code>	XLNet-RoBERTa-based	<code>intfloat/multilingual-e5-large</code> , etc.		

注意
`ssmits/Qwen2-7B-Instruct-embed-base` 的 Sentence Transformers 配置定义不正确。您应该通过传递 `--override-pooler-config '{"pooling_type": "MEAN"}'` 来手动设置均值池化。

注意

与基础版 Qwen2 不同，Alibaba-NLP/gte-Qwen2-7B-instruct 使用双向注意力。您可以通过设置 `--hf-overrides '{"is_causal": false}'` 来相应地更改注意力掩码。

另一方面，它的 1.5B 版本（Alibaba-NLP/gte-Qwen2-1.5B-instruct）尽管在模型卡上描述不同，但仍使用因果注意力。

无论是哪个版本，您都需要启用 `--trust-remote-code`，以加载正确的分词器。请参阅 HF Transformers 上的相关问题。

如果您的模型不在上述列表中，我们将尝试使用 `as_embedding_model()` 自动转换该模型。默认情况下，整个提示的嵌入将从对应于最后一个 token 的归一化隐藏状态中提取。

2.2.2 奖励模型（`--task reward`）

Architecture	Models	Example HF Models	LoRA	PP
<code>InternLM2ForRewardModel</code>	InternLM2-based	<code>internlm/internlm2-1_8b-reward</code> , <code>internlm/internlm2-7b-reward</code> , etc.	✓	✓
<code>LlamaForCausalLM</code>	Llama-based	<code>peiyi9979/math-shepherd-mistral-7b-prm</code> , etc.	✓	✓
<code>Qwen2ForRewardModel</code>	Qwen2-based	<code>Qwen/Qwen2.5-Math-RM-72B</code> , etc.	✓	✓
<code>Qwen2ForProcessRewardModel</code>	Qwen2-based	<code>Qwen/Qwen2.5-Math-PRM-7B</code> , <code>Qwen/Qwen2.5-Math-PRM-72B</code> , etc.	✓	✓

如果您的模型不在上述列表中，我们将尝试使用 `as_reward_model()` 自动转换该模型。默认情况下，我们直接返回每个 token 的隐藏状态。

重要提示

对于过程监督奖励模型，例如 `peiyi9979/math-shepherd-mistral-7b-prm`，池化配置应该明确设置，例如：`--override-pooler-config '{"pooling_type": "STEP", "step_tag_id": 123, "returned_token_ids": [456, 789]}'`。

2.2.3 分类（`--task classify`）

Architecture	Models	Example HF Models	LoRA	PP
<code>JambaForSequenceClassification</code>	Jamba	<code>ai21labs/Jamba-tiny-reward-dev</code> , etc.	✓	✓
<code>Qwen2ForSequenceClassification</code>	Qwen2-based	<code>jason9693/Qwen2.5-1.5B-apeach</code> , etc.	✓	✓

如果您的模型不在上述列表中，我们将尝试使用 `as_classification_model()` 自动转换该模型。默认情况下，类别概率是从对应于最后一个 token 的 softmax 隐藏状态中提取的。

2.2.4 句子对评分（-task score）

Architecture	Models	Example HF Models	LoRA	PP
<code>BertForSequenceClassification</code>	BERT-based	<code>cross-encoder/ms-marco-MiniLM-L-6-v2</code> , etc.		
<code>RobertaForSequenceClassification</code>	RoBERTa-based	<code>cross-encoder/quora-roberta-base</code> , etc.		
<code>XLMRobertaForSequenceClassification</code>	XLM-RoBERTa-based	<code>BAAI/bge-reranker-v2-m3</code> , etc.		

2.3 多模态语言模型的列表

以下模态根据模型的不同而有所支持：

文本

图像

视频

音频

支持任意组合的模态，用 + 连接。

例如：T+I表示模型支持仅文本、仅图像和文本与图像的输入。

另一方面，用 / 分隔的模态是互斥的。

例如：T/I表示模型支持仅文本和仅图像输入，但不支持文本与图像的输入。

请查看此页面了解如何将多模态输入传递给模型。

重要提示
为了在每个文本提示中启用多个多模态项，您必须设置 `limit_mm_per_prompt`（离线推理）或 `--limit-mm-per-prompt`（在线服务）。例如，要启用每个文本提示最多传递 4 张图像：
离线推理：

```
llm = LLM(  
    model="Qwen/Qwen2-VL-7B-Instruct",  
    limit_mm_per_prompt={  
        "image": 4},  
)
```

在线服务：

```
vllm serve Qwen/Qwen2-VL-7B-Instruct --limit-mm-per-prompt image=4
```

注意
vLLM 目前仅支持将 LoRA 添加到多模态模型的语言主干中。

2.3.1 生成模型

请查看此页面以获取有关如何使用生成模型的更多信息。

文本生成（`--task generate`）

Architecture	Models	Inputs	Example HF Models
<code>AriaForConditionalGeneration</code>	Aria	T + I ⁺	<code>rhymes-ai/Aria</code>
<code>Blip2ForConditionalGeneration</code>	BLIP-2	T + I ^E	<code>Salesforce/blip2-opt-2.7b</code> , <code>Salesforce/blip2-opt-6.7b</code> , etc.
<code>ChameleonForConditionalGeneration</code>	Chameleon	T + I	<code>facebook/chameleon-7b</code> etc.
<code>DeepseekVLV2ForCausalLM</code> [^]	DeepSeek-VL2	T + I ⁺	<code>deepseek-ai/deepseek-vl2-tiny</code> , <code>deepseek-ai/deepseek-vl2-small</code> , <code>deepseek-ai/deepseek-vl2</code> etc.
<code>FuyuForCausalLM</code>	Fuyu	T + I	<code>adept/fuyu-8b</code> etc.
<code>GLM4VForCausalLM</code> [^]	GLM-4V	T + I	<code>THUDM/glm-4v-9b</code> , <code>THUDM/cogagent-9b-20241220</code> etc.
<code>H2OVLChatModel</code>	H2OVL	T + I ^{E+}	<code>h2oai/h2ovl-mississippi-800m</code> , <code>h2oai/h2ovl-mississippi-2b</code> , etc.
<code>Idefics3ForConditionalGeneration</code>	Idefics3	T + I	<code>HuggingFaceM4/Idefics3-8B-Llama3</code> etc.
<code>InternVLChatModel</code>	InternVL 2.5, Mono-InternVL, InternVL 2.0	T + I ^{E+}	<code>OpenGVLab/InternVL2_5-4B</code> , <code>OpenGVLab/Mono-InternVL-2B</code> , <code>OpenGVLab/InternVL2-4B</code> , etc.
<code>LlavaForConditionalGeneration</code>	LLaVA-1.5	T + I ^{E+}	<code>llava-hf/llava-1.5-7b-hf</code> , <code>TIGER-Lab/Mantis-8B-siglip-llama3</code> (see note), etc.
<code>LlavaNextForConditionalGeneration</code>	LLaVA-NeXT	T + I ^{E+}	<code>llava-hf/llava-v1.6-mistral-7b-hf</code> , <code>llava-hf/llava-v1.6-vicuna-7b-hf</code> , etc.

<code>LlavaNextVideoForConditionalGeneration</code>	LLaVA-NeXT-Video	T + V	<code>llava-hf/LLaVA-NeXT-Video-7B-hf</code> , etc.
<code>LlavaOnevisionForConditionalGeneration</code>	LLaVA-Onevision	T + I ⁺ + V ⁺	<code>llava-hf/llava-onevision-qwen2-7b-ov-hf</code> , <code>llava-hf/llava-onevision-qwen2-0.5b-ov-hf</code> , etc.
<code>MiniCPMO</code>	MiniCPM-O	T + I ^{E+} + V ^{E+} + A ^{E+}	<code>openbmb/MiniCPM-o-2_6</code> , etc.
<code>MiniCPMV</code>	MiniCPM-V	T + I ^{E+} + V ^{E+}	<code>openbmb/MiniCPM-V-2</code> (see note), <code>openbmb/MiniCPM-Llama3-V-2_5</code> , <code>openbmb/MiniCPM-V-2_6</code> , etc.
<code>MllamaForConditionalGeneration</code>	Llama 3.2	T + I ⁺	<code>meta-llama/Llama-3.2-90B-Vision-Instruct</code> , <code>meta-llama/Llama-3.2-11B-Vision</code> , etc.
<code>MolmoForCausalLM</code>	Molmo	T + I	<code>allenai/Molmo-7B-D-0924</code> , <code>allenai/Molmo-7B-O-0924</code> , etc.
<code>NVLM_D_Model</code>	NVLM-D 1.0	T + I ⁺	<code>nvidia/NVLM-D-72B</code> , etc.
<code>PaliGemmaForConditionalGeneration</code>	PaliGemma, PaliGemma 2	T + I ^E	<code>google/paligemma-3b-pt-224</code> , <code>google/paligemma-3b-mix-224</code> , <code>google/paligemma2-3b-ft-docci-448</code> , etc.
<code>Phi3VForCausalLM</code>	Phi-3-Vision, Phi-3.5-Vision	T + I ^{E+}	<code>microsoft/Phi-3-vision-128k-instruct</code> , <code>microsoft/Phi-3.5-vision-instruct</code> , etc.
<code>PixtralForConditionalGeneration</code>	Pixtral	T + I ⁺	<code>mistralai/Pixtral-12B-2409</code> , <code>mistral-community/pixtral-12b</code> (see note), etc.
<code>QwenVLForConditionalGeneration</code> [^]	Qwen-VL	T + I ^{E+}	<code>Qwen/Qwen-VL</code> , <code>Qwen/Qwen-VL-Chat</code> , etc.
<code>Qwen2AudioForConditionalGeneration</code>	Qwen2-Audio	T + A ⁺	<code>Qwen/Qwen2-Audio-7B-Instruct</code>
<code>Qwen2VLForConditionalGeneration</code>	QVQ, Qwen2-VL	T + I ^{E+} + V ^{E+}	<code>Qwen/QVQ-72B-Preview</code> , <code>Qwen/Qwen2-VL-7B-Instruct</code> , <code>Qwen/Qwen2-VL-72B-Instruct</code> , etc.
<code>Qwen2_5_VLForConditionalGeneration</code>	Qwen2.5-VL	T + I ^{E+} + V ^{E+}	<code>Qwen/Qwen2.5-VL-3B-Instruct</code> , <code>Qwen/Qwen2.5-VL-72B-Instruct</code> , etc.

UltravoxModel	Ultravox	T +	fixie-ai/ultravox-v0_5-llama-3_2-1b
---------------	----------	-----	-------------------------------------

您需要通过 `--hf-overrides` 设置架构名称，以匹配 vLLM 中的架构名称。

• 例如，要使用 DeepSeek-VL2 系列模型：

```
--hf-overrides '{"architectures": ["DeepseekVLV2ForCausalLM"]}'
```

E 此模态可以输入预先计算的嵌入。

- 对于此模态，可以在每个文本提示中输入多个项。
注意

h2oai/h2ovl-mississippi-2b 在我们支持 FlashAttention 以外的后端后，将在 V1 中提供。

注意

要使用 TIGER-Lab/Mantis-8B-siglip-llama3，在运行 vLLM 时，您必须传递 `--hf-overrides '{"architectures": ["MantisForConditionalGeneration"]}'`。

注意

官方的 openbmb/MiniCPM-V-2 目前无法使用，因此我们现在需要使用一个分支版本（HwwwH/MiniCPM-V-2）。有关更多详情，请参阅：Pull Request #4087。

注意

mistral-community/pixtral-12b 目前尚不支持 V1。

注意

要使用 Qwen2.5-VL 系列模型，您必须通过 `pip install git+https://github.com/huggingface/transformers` 从源代码安装 Huggingface transformers 库。

2.3.2 池化模型

请查看此页面以获取有关如何使用池化模型的更多信息。

重要提示

由于某些模型架构同时支持生成任务和池化任务，您应该明确指定任务类型，以确保模型以池化模式而不是生成模式使用。

2.3.2.1 文本嵌入（`-task embed`）

任何文本生成模型都可以通过传递 `--task embed` 转换为嵌入模型。

注意
为了获得最佳效果，您应该使用专门训练过的池化模型。

下表列出了在 vLLM 中经过测试的模型。

Architecture	Models	Inputs	Example HF Models	LoRA	PP
LlavaNextForConditionalGeneration	LLaVA-NeXT-based	T / I	royokong/e5-v		✔
Phi3VForCausalLM	Phi-3-Vision-based	T + I	TIGER-Lab/VLM2Vec-Full	🚧	✔
Qwen2VLForConditionalGeneration	Qwen2-VL-based	T + I	MrLight/dse-qwen2-2b-mrl-v1		✔

2.3.2.2 转录（--task transcription）

专门为自动语音识别训练的 Speech2Text 模型。

Architecture	Models	Example HF Models	LoRA	PP
Whisper	Whisper-based	openai/whisper-large-v3-turbo	🚧	🚧

2.4 模型支持政策

在 vLLM，我们致力于促进第三方模型在我们生态系统中的集成和支持。我们的做法旨在平衡对稳健性的需求和支持广泛模型的实际限制。以下是我们如何管理第三方模型支持的方式：

社区驱动支持：我们鼓励社区贡献以添加新模型。当用户请求支持新模型时，我们欢迎社区提交拉取请求（PR）。这些贡献主要根据它们生成的输出的合理性来评估，而不是严格与现有实现（例如 transformers 中的实现）的一致性。征集贡献：来自模型供应商的 PR 我们将非常感激！

尽力保持一致性：虽然我们旨在保持 vLLM 中实现的模型与其他框架（如 transformers）之间的一定一致性，但完全对齐并不总是可行的。加速技术和使用低精度计算等因素可能会引入差异。我们的承诺是确保实现的模型是功能性的，并生成合理的结果。

提示

在比较 HuggingFace Transformers 中 `model.generate` 的输出与 vLLM 中 `llm.generate` 的输出时，请注意，前者会读取模型的生成配置文件（即 `generation_config.json`），并应用默认的生成参数，而后者仅使用传递给函数的参数。在比较输出时，请确保所有采样参数一致。

问题解决和模型更新：我们鼓励用户报告他们在使用第三方模型时遇到的任何错误或问题。建议的修复应通过拉取请求（PR）提交，并清楚地解释问题和提出解决方案的理由。如果一个模型的修复影响到另一个模型，我们依赖社区来突出并解决这些跨模型依赖关系。注意：对于修复错误的 PR，良好的礼仪是通知原作者，征求他们的反馈。

监控和更新：有兴趣的用户应监控特定模型的提交历史（例如，通过跟踪 `main/vllm/model_executor/models` 目录中的更改）。这种主动的方式有助于用户及时了解可能影响他们使用的模型的更新和变化。

选择性关注：我们的资源主要集中在具有显著用户兴趣和影响力的模型上。使用频率较低的模型可能会得到较少关注，我们依赖社区在其维护和改进中发挥更积极的作用。

通过这种方式，vLLM 促进了一个合作环境，核心开发团队和更广泛的社区共同贡献于我们生态系统中支持的第三方模型的稳健性和多样性。

请注意，作为一个推理引擎，vLLM 不会引入新模型。因此，vLLM 支持的所有模型在这方面都是第三方模型。

我们对模型有以下几种测试级别：

严格一致性：我们将模型的输出与 HuggingFace Transformers 库中模型在贪婪解码下的输出进行比较。这是最严格的测试。请参阅模型测试，查看通过此测试的模型。

输出合理性：我们检查模型的输出是否合理且连贯，通过测量输出的困惑度并检查是否存在明显的错误。这是一个较为宽松的测试。

运行时功能性：我们检查模型是否可以加载并运行而不出错。这是最宽松的测试。请参阅功能性测试和示例，查看通过此测试的模型。

社区反馈：我们依赖社区提供关于模型的反馈。如果模型损坏或无法按预期工作，我们鼓励用户报告问题或打开拉取请求进行修复。其余的模型属于此类别。



AI 专题精讲

已关注



专栏目录

已订阅